5 System Implementation, Testing and Evaluation

5.1 Introduction

This chapter explains how the design approved in Chapter 4 was implemented, the environment in which it was built and deployed, and the evidence that the implementation works. Because the project is an AI-based information system, the chapter follows the route that links the database, the dataset, the model and the application. Section 5.2 describes the implementation environment. Section 5.3 describes the system implementation, from the database and the dataset to model training and the recommendation service, and records where the implementation departed from the design. Section 5.4 covers testing and evaluation, Section 5.5 the version-control evidence, Section 5.6 the use of AI-assisted tools, and Section 5.7 the evaluation results, discussion and limitations.

Unless stated otherwise, every figure in Sections 5.2 and 5.3 was reproduced from the project repository using a clean PostgreSQL and PostGIS container built from the schema file and the migration script. The training data are synthetic, so the results describe the behaviour of the system on simulated Nairobi data and must not be read as measurements of real providers. The evidence in Section 5.4 was collected from the committed repository together with the administrator functions, which were in the working tree and not yet committed, as explained in Section 5.5.

5.2 Implementation Environment

The implementation environment describes where the solution was developed, tested and deployed, and what is needed to reproduce it. The development environment is distinguished from the hosted environment, which is a free-tier demonstration deployment and not a production service.

5.2.1 Hardware and Deployment Environment

The system was developed, trained and tested on a single laptop. The model is small enough (3,000 records and five features) to be trained on the CPU without an accelerator. For demonstration, the three tiers of the architecture were deployed to hosted services, each in the tier that suited its workload: the web application on Vercel, the FastAPI and XGBoost service in

a Docker container on Render, and the database on Neon. The Render and Neon instances were placed in Frankfurt to keep the two close to each other. Table 5.7 summarises the environment.

Table 5.7 Implementation environment

Layer	**Specification tested**	**Purpose**	**Evidence or source**
Development workstation	AMD Ryzen 5 5625U, 12 logical CPUs, 14 GiB memory, Ubuntu 24.10, Docker 29.1.3	Development, model training and automated tests	lscpu, free and /etc/os-release on the workstation
Local database	postgis/postgis:16-3.4 container (PostgreSQL 16, PostGIS 3.4)	Spatial and relational storage during development	SELECT PostGIS_Version() returned 3.4
Hosted API	Render web service, Docker runtime, Frankfurt region, free plan, 512 MB memory; sleeps after 15 minutes idle, so the first request after a pause takes about 30 to 60 seconds	Runs the FastAPI service and XGBoost inference	render.yaml; docs/DEPLOYMENT.md
Hosted database	Neon PostgreSQL 16 with PostGIS, AWS eu-central-1 (Frankfurt)	Hosted data store for the deployed API	docs/ DEPLOYMENT.md
Hosted frontend	Vercel project with root directory frontend, API address set through NEXT_PUBLIC_API_URL	Serves the Next.js web application	docs/ DEPLOYMENT.md
Client device	Current desktop and mobile browsers. Automated tests drive desktop Chrome through Playwright. A minimum phone specification was not measured.	Runs the web application	tests/e2e/ playwright.config.ts

5.2.2 Software, Services and Configuration

Table 5.8 lists the software that was used to build and run the system, with the versions that were tested. The Python dependencies are pinned to exact versions in two files: requirements.txt for development and requirements-api.txt for the deployed image, which omits the demonstration, plotting and testing packages. The frontend versions are those declared in package.json, with the installed version of Next.js shown where it differs. Passwords, database credentials and other secrets are held in a local file that is excluded from version control and in the Render and Vercel dashboards, and none appears in the repository.

Table 5.8 Software, libraries and services

Layer	**Technology and version**	**Role in the system**
Language and runtime	Python 3.12.7; Node.js 22.12.0	Backend and machine learning code; frontend build and tooling
API	FastAPI 0.115.0 with Uvicorn 0.30.6	Serves the recommendation, booking, account and notification endpoints
Machine learning	XGBoost 2.1.1; scikit-learn 1.5.2; pandas 2.2.3; joblib 1.4.2	Feature handling, the encoding step and regressor of the pipeline, and model persistence
Database	PostgreSQL 16 with PostGIS 3.4; psycopg2-binary 2.9.9	Relational and spatial storage
Security	bcrypt 4.2.0	Password hashing
Frontend	Next.js 14.2.35 (declared ^14.2.15); React ^18.3.1; TypeScript ^5.5.4; Tailwind CSS ^3.4.13	Web application, routing and styling
Maps	Leaflet 1.9.4; react-leaflet 4.2.1; OpenStreetMap tiles	Location picker and map display
Testing	pytest 8.3.3 with httpx 0.27.2; Playwright 1.59.1	Backend tests; end-to-end browser tests

Layer	Technology and version	Role in the system
Delivery	Docker; Git and GitHub; Vercel, Render and Neon	Containers, version control and hosting

5.2.3 Deployment and Reproducibility

The system can be rebuilt from the repository with the following steps. Steps 1, 2 and 5 and the backend test suite were carried out against a new, empty PostGIS container to confirm that the schema, the migration and the tests work from a clean start. The model figures in Section 5.3 were reproduced with the same pipeline definition that Step 4 uses. Table 5.9 lists the steps.

Table 5.9 Steps to rebuild the system from the repository

Step	Component	Action and evidence
1	Database	Start a postgis/postgis:16-3.4 container and apply database/schema.sql, which enables PostGIS and creates the thirteen application tables.
2	Data	Run database/migrate_csv_to_postgres.py with the DB_HOST, DB_PORT, DB_NAME, DB_USER and DB_PASSWORD variables set. The script loads the source CSV files into the reference and training tables and can be repeated with its --reset option.
3	Administrator	Run database/seed_admin.py with the ADMIN_EMAIL and ADMIN_PASSWORD variables to create the only kind of account that cannot be registered through the interface.
4	Model	Install the pinned dependencies and run finalize_model.py in ml-service. It trains the pipeline with a fixed random seed, prints the test and cross-validation errors and saves the model file. The Docker image used for the hosted API runs the same script during its build.
5	Api	Start the service with Uvicorn and call GET /health. The check returned {"status":"ok","providers_loaded":200,"areas_loaded":30,"availability_loaded":1117}, which confirms that the model and the reference data were loaded.

Step	Component	Action and evidence
6	Frontend	Run npm install and npm run dev in the frontend folder with NEXT_PUBLIC_API_URL pointing to the API.

The additive migrations 002 to 007 in database/migrations upgrade a database that was created from an earlier version of the schema; a fresh installation uses schema.sql, which already contains their changes. With the database variables set, 182 backend tests passed and 13 were skipped by default. With RUN_DB_TESTS=1 the 13 PostgreSQL-isolated tests also ran, and all 195 backend tests passed. The suite was run again after the administrator functions were added, and all 205 backend tests passed.

5.3 System Implementation

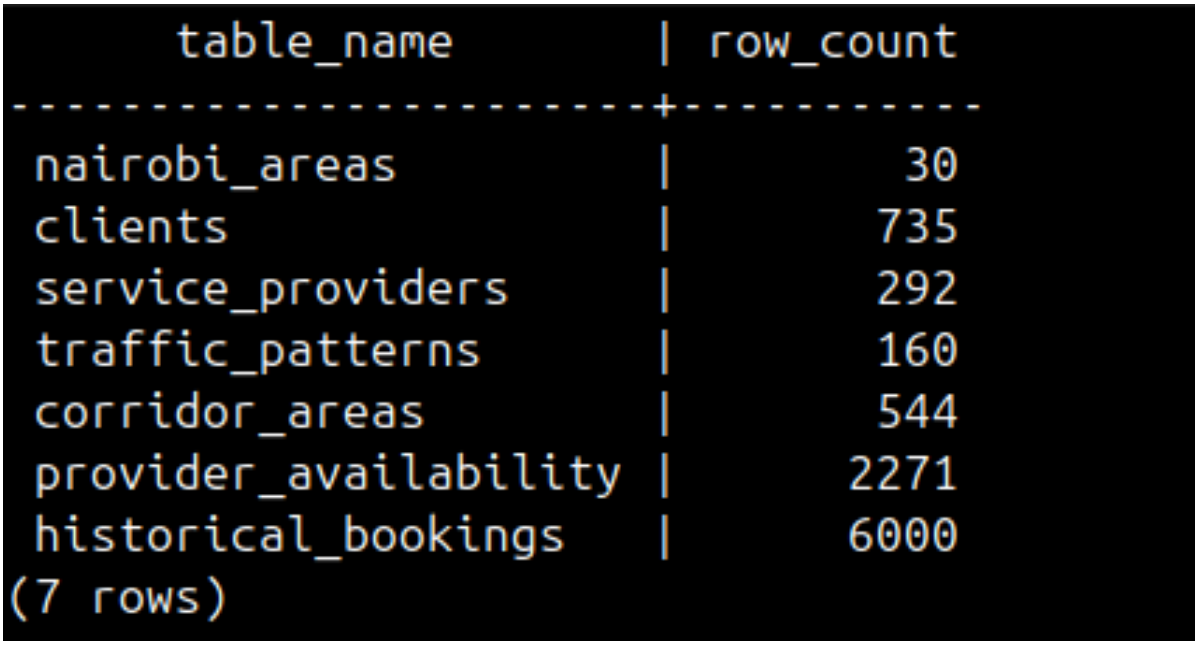
The implementation is described in the order of the architecture in Chapter 4: first the database, then the dataset and the model, and then the service that combines them. Each subsection identifies the requirements it implements and the evidence that it works.

5.3.1 Database and Data Implementation

The logical schema of Section 4.4.4 was implemented in PostgreSQL 16 with the PostGIS extension. The schema file creates thirteen application tables, GIST indexes on the point geometries of areas, clients and providers, and B-tree indexes on the foreign keys that the queries join on. Later requirements were met by additive migrations instead of changes to the original tables: exact locations (002), booking reviews (003), cancellation reasons (004), job dates (005) reschedule proposals (006) and provider identity reviews (007). Each migration can be applied again without harm.

The CSV files were loaded by the migration script. The pipe-delimited list of served areas in the traffic file was split into rows of the corridor_areas junction table, so that 160 traffic patterns produced 544 corridor and area pairs. Figure 5.17 shows the current PostgreSQL row counts captured from the running database: 30 areas, 735 clients, 292 service providers, 160 traffic patterns, 544 corridor-area mappings, 2,271 availability records and 6,000 historical bookings. These are the database contents at screenshot capture time, distinct from the original 3,000-record training dataset described in Section 5.3.2. The screenshot alone does not establish why the counts have increased or that the model was retrained on those additional rows. The tables

that hold application activity (users, bookings, notifications, booking_reviews and booking_reschedules) start empty and are filled only by real use of the application, in line with DR-05 and DR-06.



table_name	row_count
nairobi_areas	30
clients	735
service_providers	292
traffic_patterns	160
corridor_areas	544
provider_availability	2271
historical_bookings	6000

(7 rows)

Figure 5.17 Current database row counts from PostgreSQL

The integrity rules of requirement NFR-04 were checked by attempting to break them directly in the database. Inserting a user with the role client and a provider identifier failed with a violation of the constraint chk_users_role_matches_fk, and inserting a historical booking with a reliability score of 1.5 failed with a violation of the check constraint on arrival_reliability_score. In both cases the database rejected the row without any help from the application, which means that a defect in the API cannot store an inconsistent account or an impossible score.

5.3.2 Dataset Description

The training data are the 3,000 synthetic bookings in historical_bookings. They were generated by the rule-based simulation described in Section 3.2.1, with congestion, speeds and road quality calibrated to published information about Nairobi. Each record describes one booking: who booked whom, the service type, the day and time slot, the distance between the two locations, the congestion multiplier of the provider’s corridor, the road quality of the areas, the provider’s rating, completion rate and experience, the outcome of the job, and the arrival reliability score

that is the target of the model. Table 5.10 summarises the dataset and Figure 5.18 shows three records with the five model features and the target.

Table 5.10 Summary of the training dataset

Property	**Value**
Source and licence	Synthetic, generated for this project from a rule-based simulation. It contains no real persons or bookings.
Size	6,000 records with 36 columns, booked between 1 January 2024 and 23 June 2025
Entities	200 providers, 499 of the 500 clients, 30 areas and 160 traffic patterns
Service types	electrician 719; cleaner 661; plumber 639; technician 568; carpenter 413
Time slots	midday 817; early_afternoon 605; evening_rush 593; mid_morning 508; morning_rush 477
Day types	weekday 2,120; weekend 880
Road quality of the provider's area	good 1,076; poor 1,020; moderate 904
Distance (km)	mean 13.72; median 11.91; range 0.28 to 46.20
Congestion multiplier	mean 2.37; range 1.04 to 4.00
Target: arrival_reliability_score	mean 0.765; standard deviation 0.189; median 0.814; range 0.0 to 1.0. 9.7% of records are below 0.5 and 28.8% are at or above 0.9.
Data quality	No missing values in the five model features or the target; no duplicate booking identifiers

```
SOURCE: data/raw/historical_bookings.csv
First three records — selected model inputs and target

booking_id                    B00001          B00002        B00003
service_type                  plumber         plumber    technician
distance_km                     10.73           24.83         18.43
congestion_multiplier            2.02            1.94           4.0
time_slot                 morning_rush  early_afternoon  evening_rush
provider_area_road_quality        good            poor          poor
provider_completion_rate         0.977           0.904         0.809
arrival_reliability_score       0.9744          0.7068        0.3562
```

Figure 5.18 Actual training dataset preview: model features and target

The sample shows the pattern that the model has to learn. The third record combines a long distance, the highest congestion multiplier and a poor-quality area in the evening rush, and it has the lowest score, while the first combines a good area, moderate congestion and a high completion rate and has a score close to 1. The data cover five service types. The approved proposal named three; technician and carpenter were added in the simulation and are offered by the system, and Section 1.5.1 has been updated to match.

5.3.3 Data Preparation and Partitioning

The model uses five input features, one for each conceptual pillar of the design: distance_km for spatial separation, congestion_multiplier for traffic, time_slot for time of day, provider_area_road_quality for road quality, and provider_completion_rate for provider history. The set was reduced from a larger candidate set after testing indicated that the five features matched the performance of the larger set. The estimated travel time is computed for display and explanation but is not a model input, because it is a function of distance and speed that the model already receives.

In line with DR-02, the columns that are known only after a job has ended (actual_travel_min, delay_min, on_time, job_completed and client_rating_given) were excluded, since using them would leak the outcome into the prediction. The data had no missing values in the model columns, so no imputation was needed for training. Preprocessing is part of the saved pipeline: the numeric features pass through unchanged and the two categorical features are one-hot encoded, with unseen categories ignored. Because the encoder is inside the pipeline, it is fitted on training data only, including inside each cross-validation fold.

The records were split at random into 2,400 for training (80%) and 600 for testing (20%) with a fixed seed of 42. The 600 test records were used only for the final comparison and not for tuning. Hyperparameters were selected by five-fold cross-validation on the training portion. The split is random across bookings and not grouped by provider, so the same providers appear in both parts. The test therefore measures how well new bookings of known providers are predicted, and it does not measure how well the model generalises to providers that it has never seen.

5.3.4 Model Training and Selection

Three conditions were compared, as set out in Section 3.2.3. Baseline A predicts the score from distance alone and Baseline B from the provider's rating alone, in both cases with a linear regression fitted on the training records. The proposed model is an XGBoost regressor inside the scikit-learn pipeline. A grid search with five-fold cross-validation over the learning rate (0.05 and 0.1), the maximum tree depth (3 and 5) and the minimum child weight (1 and 3) selected a learning rate of 0.05, a depth of 3 and a minimum child weight of 3. The other parameters kept their defaults and the random seed was 42. The untuned model, trained with default parameters, obtained an RMSE of 0.1583 and an MAE of 0.1247 on the test set in the training notebook, so tuning reduced the error further.

Table 5.11 compares the conditions on the same 600 test records. The proposed model has an RMSE of 0.1417 and an MAE of 0.1160, which are 24.3% and 23.4% lower than those of Baseline A and 26.8% lower than those of Baseline B on both measures. Its R^2 is 0.4717, against 0.0776 for distance alone and 0.0141 for rating alone. The target is continuous, so RMSE, MAE and R^2 are the appropriate measures and accuracy is not used. These results satisfy requirement NFR-01 on this dataset.

Table 5.11 Model comparison on the 600-record test set

Condition	**RMSE**	**MAE**	**R^2**
Baseline A: distance only (linear regression)	0.1872	0.1514	0.0776
Baseline B: provider rating only (linear regression)	0.1936	0.1585	0.0141

Condition	RMSE	MAE	R^2
XGBoost with default parameters (training notebook)	0.1583	0.1247	Not reported
Proposed model: tuned XGBoost, five features	0.1417	0.1160	0.4717

Five-fold cross-validation of the final pipeline over all 3,000 records gave a mean RMSE of 0.1414 with a standard deviation of 0.0020, and the fold errors ranged from 0.1387 to 0.1449. The mean is almost the same as the test RMSE, which indicates that the result does not depend on a favourable split. An earlier verification record shows a test RMSE of 0.1419 and an MAE of 0.1162, and the notebook reports a cross-validation mean of 0.1411. These differ from the values above only in the fourth decimal place and reflect small variation between runs and environments.

Table 5.12 Relative importance of the model features (gain)

Feature	Share of total importance
congestion_multiplier	51.5%
provider_completion_rate	16.5%
time_slot (five encoded values)	11.9%
provider_area_road_quality (three encoded values)	10.3%
distance_km	9.8%

The importance values in Table 5.12 answer the first research question for this dataset: congestion on the provider's corridor carries about half of the model's explanatory weight, provider history comes second, and straight-line distance, the factor on which current platforms rank providers, contributes under a tenth. This agrees with the low R^2 of the distance baseline. The values should not be read as causal effects in Nairobi, because the data were produced by a simulator in which these relationships were built in, and gain-based importance depends on how the features are correlated.

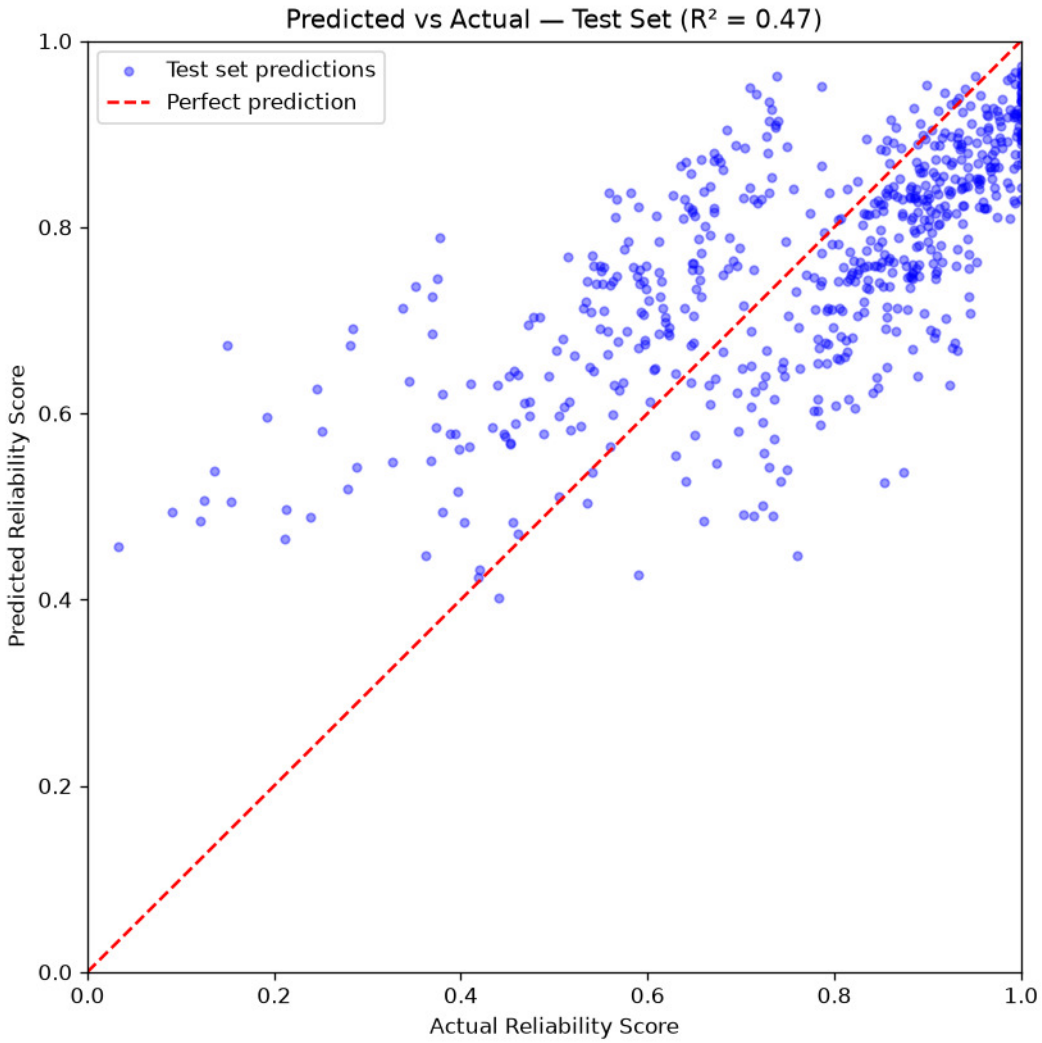


Figure 5.19 Predicted against actual reliability score on the test set

Figure 5.19 plots the predictions for the 600 test records against the actual scores. The points follow the diagonal, which confirms that the model has learned the trend, but they are spread widely around it. Records with a low actual score lie mostly above the line, so the model over-predicts the reliability of the least reliable bookings and compresses its predictions into a narrower range than the actual scores. An R^2 of 0.47 means that about half of the variance in the target is explained. The remainder reflects the random variation that the simulation adds to each outcome and any influences that are not among the five features.

5.3.5 Recommendation Service

The recommendation service in api.py combines the database, the feature engineering module and the saved model. When the service starts it loads the model and the reference tables into memory. A POST /recommend request is first validated, so that a time slot outside the four offered, a day type that contradicts the job date, or a date in the past or more than 60 days ahead is rejected with status 422. Providers of the requested service type without an availability record for the weekday and slot are removed, and a feature row is built for every remaining provider. The model scores each row, the providers are sorted by score, a rule-based explanation is attached, and the optional filters on rate, rating and verification are applied before the list is cut to the requested length. A separate endpoint lists new providers so that a provider with no history is not hidden (FR-08), and the booking endpoint recomputes the score on the server (FR-09).

Figure 5.20 shows the saved client interface for an electrician request in Kilimani during the evening rush on Monday 2 November 2026. The visible results rank Rose Ali first with a displayed reliability score of 78%, a distance of 8.41 km and an estimated travel time of 37.1 minutes, followed by Adhiambo Kamau at 77%, 9.75 km and 38.2 minutes. Each card displays a rate, rating and explanation. The explanation identifies heavy congestion, good road access and a strong historical completion rate. This interface capture is a separate observation from the earlier API example; the displayed values are rounded. The screenshot also appears in Figure A2.25; Figures A2.26 and A2.5 show the subsequent booking screens.

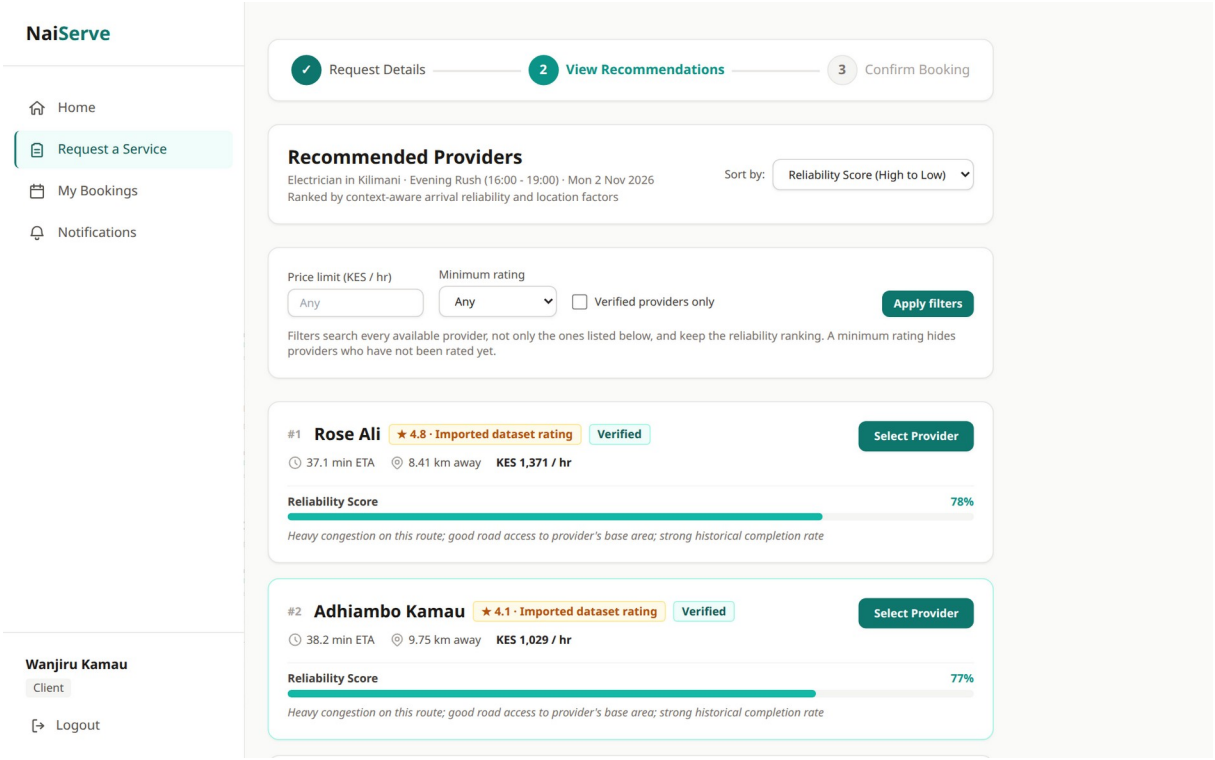


Figure 5.20 Recommendation results in the working client interface

5.4 Testing and Evaluation

This section reports whether the implemented system satisfies the requirements of Chapter 4. It follows the order of test strategy, test cases and defects, project-specific evaluation and interpretation. Every result was produced on the development workstation of Table 5.7, against the local PostGIS container.

5.4.1 Test Strategy and Acceptance Criteria

The purpose of testing was to show that each requirement of Chapter 4 that has a verification method is met by the implementation, and that the model satisfies NFR-01. The scope is the FastAPI service with its database, the model and the browser interface. Four levels were used because each answers a different question (Table 5.13). The test data are the synthetic reference data of Section 5.3.2 and the accounts and bookings that the tests create for themselves, so no data about a real person were used. The author ran the tests, and the completion criterion for the automated levels is that every test passes when the documented environment variables are set.

Table 5.13 Test levels, tools and acceptance criteria

Level	**Question answered**	**Tool and scope**	**Acceptance criterion**
Unit	Do feature computation and input validation return the right values?	pytest 8.3.3; test_feature_engineering.py and the validation tests	Every test passes
Integration	Do the API, the database and the model work together, including error responses?	pytest with the FastAPI test client (httpx 0.27.2); 205 tests in 15 files; tests that need a live database run only with RUN_DB_TESTS=1 against an isolated PostGIS database	Every test passes
System	Can a client and a provider complete their flows in a browser?	Playwright 1.59.1; 20 tests in 12 files in tests/e2e	Every test passes on a run against the running stack
Model experiment	Does the model beat both baselines on unseen data?	Held-out 600-record test set (Section 5.3.4)	Lower RMSE and MAE than both baselines (NFR-01)
Performance	How long does a recommendation request take?	200 sequential requests (Section 5.4.3)	A limit is fixed after benchmarking the deployed environment (NFR-05)

5.4.2 Test Cases, Results, Defects and Retesting

Table 5.14 summarises the 205 automated backend tests by area and traces each area to the requirements it verifies. The test cases themselves are the functions in tests/ml-service, whose names state the scenario and whose assertions state the expected result, so another evaluator can repeat them with the command in Section 5.2.3. With RUN_DB_TESTS=1 and the database variables set, all 205 tests passed in 15.98 seconds with 7 warnings. The warnings that were

displayed are a pandas FutureWarning raised when a newly registered provider is appended to the in-memory provider table, and they do not affect any result.

Table 5.14 Automated backend tests by area

Area (test file)	**Tests**	**Requirements verified**	**Result**
Core endpoints (test_api_core)	10	FR-03, FR-05, FR-06, IR-05	All passed
Context validation (test_api_context_validation, test_past_time_slots)	10	DR-04	All passed
Feature engineering (test_feature_engineering)	9	DR-02, DR-03, FR-05	All passed
Registration and login (test_api_auth)	8	FR-01, FR-02, NFR-02	All passed
Availability (test_api_availability)	12	FR-04, FR-12	All passed
Result filters (test_api_filters)	13	FR-07	All passed
New providers (test_cold_start)	13	FR-08, DR-06	All passed
Bookings and status changes (test_api_bookings)	48	FR-09, FR-10, FR-14, IR-05	All passed
Dated scheduling (test_api_scheduling)	11	FR-09, DR-04	All passed
Rescheduling (test_api_rescheduling)	34	FR-11	All passed
Reviews (test_api_reviews)	12	FR-13, DR-06	All passed

Area (test file)	Tests	Requirements verified	Result
Profile and notifications (test_api_profile_notifications)	8	FR-14, FR-15	All passed
Exact location (test_exact_location)	13	FR-15, IR-04	All passed
Administrator functions (test_api_admin)	4	FR-16, FR-17, NFR-03	All passed
Total	205		All passed

Positive tests show that valid input is accepted. The negative and boundary cases in Table 5.15 matter more for correctness because they show that invalid input is refused with the status code that IR-05 prescribes. The two database cases were carried out by hand in SQL, as described in Section 5.3.1; the others are automated tests.

Table 5.15 Selected negative and boundary test cases

ID	Requirement	Scenario	Expected result	Actual result and verdict
TC-01	DR-04, IR-05	POST /recommend with a time slot or day type that does not exist	Rejected with 422	422 returned. Pass
TC-02	DR-04	POST /recommend for a morning slot that has already ended today, at the boundary minutes	Request rejected	Rejected at and after the boundary; current and future slots accepted. Pass
TC-03	FR-01	Register with an email address that already has an account	Rejected with 409	409 returned. Pass

ID	Requirement	Scenario	Expected result	Actual result and verdict
TC-04	FR-01	Register a provider without the provider-specific fields	Rejected with 400	400 returned. Pass
TC-05	FR-02	Log in with a wrong password; log in with an unknown email	Both rejected with 401	401 in both cases. Pass
TC-06	FR-12	Save availability with invalid or missing slots; save for an unknown provider	422 for invalid or missing slots; 404 for an unknown provider	As expected. Pass
TC-07	FR-17, NFR-03	A non-administrator requests the administrator provider list	Rejected with 403	403 returned. Pass
TC-08	FR-17	Request model retraining while a retraining job is already active	No second job is started	Single active job enforced. Pass
TC-09	NFR-04	Insert a user with role client and a provider identifier	Rejected by the database	Violation of chk_users_role_matches_fk. Pass

ID	Requirement	Scenario	Expected result	Actual result and verdict
TC-10	NFR-04, DR-01	Insert a historical booking with a reliability score of 1.5	Rejected by the database	Violation of the check constraint on arrival_reliability_score . Pass

Table 5.16 records the defects found while the evidence for this chapter was collected, including a failed run, which is reported and not hidden because a later run passed.

Table 5.16 Defects found, corrections and retests

ID	Defect	Cause	Correction and retest
D-01	The first run of the backend suite gave 172 passes, 10 failures and 13 skips.	The database environment variables were not set, so the ten database-dependent tests could not connect.	The variables were set as documented in the README. The rerun passed 195 tests, and 205 after the administrator tests were added.
D-02	The landing and registration pages described the recommendations as using “real” or “live” traffic.	The wording overstated the data: the system uses stored corridor and time-slot averages and has no live traffic feed.	The wording was changed to “typical traffic”. A search of the frontend source for both phrases found no match and the TypeScript check passed.
D-03	The saved model verification note reported a test RMSE of 0.1419 and an MAE of 0.1162.	The note recorded an earlier run, and the current pinned pipeline gives slightly different values.	The note was updated to 0.1417 and 0.1160, the values reproduced with the same split and seed.

5.4.3 Model, Performance and Interface Evaluation

Model evaluation. The model was compared with two baselines on the same 600 held-out records, and its variation was measured across five folds (Section 5.3.4). Retraining the model with the same data, split and seed reproduced the test RMSE of 0.1417, the MAE of 0.1160 and

the R^2 of 0.4717, and the five shuffled folds gave a mean RMSE of 0.1414 with a standard deviation of 0.0020. The training RMSE of 0.1316 is 0.010 below the test RMSE, so the model fits the training records only slightly better than unseen ones. A breakdown of the errors by condition is given in Section 5.7.2.

Performance. NFR-05 asks for a response-time limit that is fixed by benchmarking. A benchmark was run on the development workstation by sending 200 sequential POST /recommend requests for an electrician in Kilimani, on a weekday evening rush, with the top five results, after one warm-up request. The client ran in the same process as the API, so network time is not included, and the database was the local PostGIS container. Figure 5.21 shows the new run captured on 6 November 2026: all 200 measured requests succeeded, with a median of 127.4 ms, a 95th percentile of 130.6 ms and a maximum of 190.4 ms. The hosted deployment on Render and Neon was not benchmarked, and no limit has yet been fixed; NFR-05 is therefore only partly evidenced.

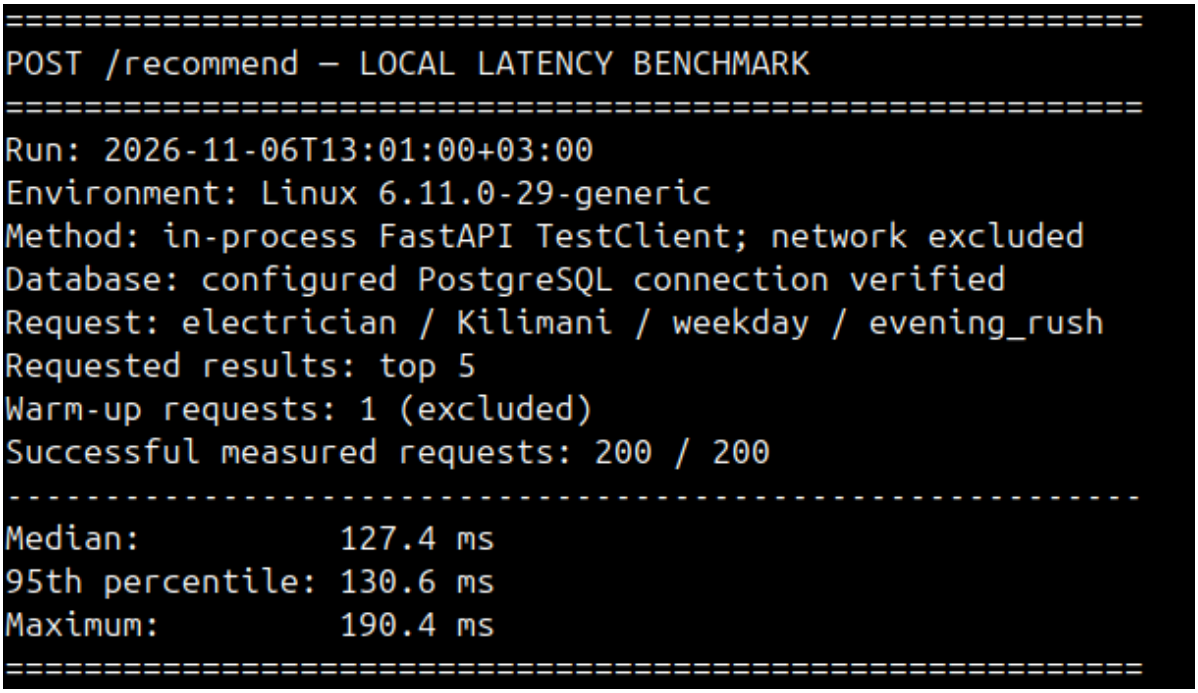


Figure 5.21 Latency of POST /recommend on the development workstation (200 requests)

Interface. The Next.js production build compiled, passed type checking and generated 21 routes in the documentation audit, and the TypeScript check passed again after the wording change in D-02. The Playwright suite contains 20 tests in 12 files that cover registration and login, the client and provider flows, scheduling, rescheduling, cancellation, reviews, exact location, result

filters, new providers, error states and installability. The suite was not rerun for this chapter because every run creates accounts and bookings that are not removed, and the development data were kept. The stored report predates the administrator functions and this chapter, so no pass count for the browser tests is claimed here. Screenshots of the client, provider and administrator screens are in Appendix A2 (Figures A2.23 to A2.8).

Human-centred evaluation. No usability test with users, accessibility audit or security test such as the OWASP Web Security Testing Guide has been carried out. The informal usability check planned for the project (issue 15) is still open. NFR-06 is supported by the fact that every recommendation carries an explanation string (Section 5.3.5), but whether clients understand and trust it has not been measured.

5.4.4 Interpretation of the Test Results

The results show that the service behaves as specified for the cases that were tested, that invalid input is refused with the intended status codes, that the database protects its own integrity and that the model result can be reproduced. They do not show that the interface is easy to use, that the service is fast under load or in the hosted environment, or that the system is secure, because those were not tested. A passing test demonstrates the absence of failure for the tested case only, and the suite was written by the author who wrote the code, so it is likely to share the author’s assumptions.

5.5 Version Control and Implementation Evidence

The project repository is at https://github.com/Theedxctor/nairobi-service-recommender and is public, so the supervisor can read the code and the history. At the time of writing the history held 106 commits, 54 issues of which 41 were closed, and 43 pull requests of which 42 were merged. Issues were assigned to the milestones M1 to M8, which run from the ML pipeline to the final documentation and the marketplace follow-ups.

Once the workflow was adopted, every change followed one workflow: an issue, a branch named feat, fix, docs or refactor with a short description, commits in the conventional format with the issue number in the footer, and a pull request that was merged with a merge commit and not squashed, so that branch and pull-request structure stays visible in the history. The pull request was the point at which the author reviewed a change before approving it. Commits made before

that point went directly to the main branch and the history was not rewritten, so the early part of the history does not follow the workflow. Table 5.17 lists the artefacts that make the work reproducible.

Table 5.17 Reproducibility artefacts in the repository

Artefact	**Purpose**
README.md	Problem, setup, configuration, run and test instructions, and the current model results
database/schema.sql; database/migrations (002 to 007)	Full schema and the additive upgrades for databases created earlier
database/ migrate_csv_to_postgres.py; database/seed_admin.py	Idempotent data load with a --reset option; the only way to create an administrator account
ml-service/requirements.txt; requirements-api.txt; frontend/package.json	Exact Python versions for development and for the deployed image; frontend dependencies
ml-service/finalize_model.py	Trains the pipeline with a fixed seed and writes ml-service/results/latest_model_metrics.json
ml-service/results/ model_performance_report_2026-10-27.md	Training, validation and test results with the protocol used
render.yaml; docs/DEPLOYMENT.md	Configuration of the hosted API, database and frontend
tests/ml-service; tests/e2e	Automated backend tests and browser tests

Secrets are kept out of the repository: environment files are excluded by .gitignore and the hosted credentials are held in the Render and Vercel dashboards. Three gaps remain. The repository has no licence file, no release tag marks the submitted version, and no deployed web address is recorded in it, so the hosted deployment was not independently verified for this report. The evidence in this chapter refers to the committed repository plus the administrator functions,

which were in the working tree and not yet committed when it was collected; they should be committed and tagged before submission so that the report can be matched to an exact version.

5.6 Author Oversight and Use of AI-Assisted Tools

Claude Code, a coding assistant from Anthropic, was used during the implementation and testing of the system. It worked under written project rules that forbid fabricated data and claims without evidence. It assisted with refactoring backend and frontend code, running the tests and the model reproduction. The product itself contains no generative AI component, the recommendation model is a gradient-boosted regression trained on tabular data.

Its output was checked in four ways: figures in the report were traced to the commands that produced them (the test run, the model reproduction, the latency benchmark and SQL queries on the database); code changes were reviewed through pull requests before merging; the report was compared against the source files in a documentation audit; and statements that could not be supported were removed or reworded, as in defects D-02 and D-03. The author made the final decisions and takes full responsibility for the submitted work.

5.7 Evaluation Results, Discussion and Limitations

This section organises the evidence by research objective, discusses why the model behaved as it did and where it fails, reports the status of the requirements, and states the limitations. The conclusions, recommendations and future work are in Chapter 6.

5.7.1 Results by Objective

Table 5.18 Results by research objective

Objective	Output and acceptance criterion	Observed result	What the result permits
i. Identify the factors that influence reliability	Five features, one per pillar, selected so that they match the larger candidate set (5.3.3); feature importance reported	Congestion 51.5%, completion rate 16.5%, time slot 11.9%, road quality 10.3%, distance 9.8% (Table 5.12)	In this synthetic dataset congestion matters most and distance least. The ranking is not evidence of causes in Nairobi.
ii. Review existing platforms and identify gaps	Chapter 2 gap analysis of Mama Fua, Fundis and Kazi Konekt	None of the reviewed platforms was found to use traffic, time-of-day or road-quality context	A gap among the reviewed platforms. It is not a claim about every platform in Kenya.
iii. Develop a model that predicts arrival reliability	Pipeline served by the API (FR-03 to FR-06, IR-03); all related tests pass	Working service with 205 passing backend tests; example ranking in Figure 5.20	A working prototype that scores providers on simulated data. It predicts a synthetic reliability score and not a real arrival time.
iv. Evaluate against distance-based and rating-based approaches	Lower RMSE and MAE than both baselines on the same test set (NFR-01)	RMSE 0.1417 against 0.1872 and 0.1936; MAE 0.1160 against 0.1514 and 0.1585; R^2 0.4717 against 0.0776 and 0.0141	The model is better on this dataset. The result depends on how the simulator was built.

Objective	Output and acceptance criterion	Observed result	What the result permits
General objective: a working system for clients and providers	Requirements FR-01 to FR-17 implemented and tested	All 17 functional requirements have an implementation. Administrator verification is a profile and contact review and not a check of government identity.	A demonstrable prototype. It is not production-ready because authentication is not verified on the server.

The fourth research question asks about improvement in recommendation accuracy. The study measured the error of the predicted reliability score and not the quality of the ranked lists, because there are no real bookings from which a correct ranking could be known. The result therefore shows that the score is closer to the simulated outcome than the baselines are, and it does not show that clients receive better providers.

5.7.2 Discussion and Error Analysis

The model beat both baselines by a wide margin because, in the simulated data, reliability depends mainly on congestion, provider history and their interaction with the time of day, and a straight line through distance or rating cannot represent that. The comparison should be read with care for two reasons. The baselines are linear regressions on one variable, so the gap combines the effect of the richer feature set with the effect of a non-linear model, and a linear model on the five features, or an XGBoost model on distance alone, was not run to separate the two. The untuned XGBoost model (RMSE 0.1583) lies between the baselines and the tuned model, so tuning accounted for a further 10.5% reduction in RMSE.

Table 5.19 shows where the errors are on the 600 test records. They are largest where the system is needed most: in the evening rush, under heavy congestion and on poor roads. The clearest pattern is that the model is least accurate for the least reliable bookings. For the 60 records whose actual score is below 0.5, the RMSE is 0.2589 and the model over-predicts by 0.226 on average, while it under-predicts the most reliable bookings by 0.086. This is the compression towards the mean that was visible in Figure 5.19, and it means that a risky booking can be ranked higher than it deserves.

Table 5.19 Test-set error by condition (actual minus predicted score)

Condition	Records	RMSE	Mean error
Time slot: mid_morning	109	0.1178	+0.0204
Time slot: midday	164	0.1281	+0.0106
Time slot: early_afternoon	123	0.1326	+0.0106
Time slot: morning_rush	109	0.1591	−0.0127
Time slot: evening_rush	95	0.1752	−0.0182
Road quality: good	225	0.1247	+0.0104
Road quality: moderate	179	0.1418	−0.0007
Road quality: poor	196	0.1589	−0.0003
Congestion multiplier ≤ 1.5	112	0.1110	+0.0178
Congestion multiplier 1.5 to 2.5	313	0.1312	+0.0098
Congestion multiplier > 2.5	175	0.1737	−0.0167
Actual score below 0.5	60	0.2589	−0.2260
Actual score 0.5 to 0.8	208	0.1425	−0.0620
Actual score above 0.8	332	0.1069	+0.0861

Two further observations follow from the results. First, the three scores in Figure 5.20 lie within 0.024 of each other, which is far smaller than the RMSE of 0.14, so the order of providers whose scores are this close is not reliable and clients should be shown the score and the explanation and not only the rank. Second, no weekend_day booking appears in the training data, so the scores for that slot are extrapolations. The explanation text names the bands of the factors and does not measure their individual contributions, which limits what NFR-06 achieves.

5.7.3 Requirement Coverage

All functional requirements FR-01 to FR-17 have an implementation and an automated or inspection-based verification method, as shown in Table 4.1 and Table 5.14. Table 5.20 gives the status of the non-functional requirements, where the evidence is uneven.

Table 5.20 Status of the non-functional requirements

ID	Status	Evidence
NFR-01	Met	Lower RMSE and MAE than both baselines on the 600-record test set (Table 5.11)
NFR-02	Met in the tested paths	Only bcrypt hashes are stored (password_hash column); the eight authentication tests pass
NFR-03	Partly met	Role checks guard the interface and administrator endpoints return 403 to other roles, but identity is not verified on the server (issue 84)
NFR-04	Met	Both constraint-violation checks were rejected by the database (TC-09, TC-10)
NFR-05	Partly evidenced	Median 127.4 ms on the workstation (Figure 5.21); no limit fixed; hosted environment not benchmarked
NFR-06	Partly met	Every recommendation carries an explanation; no user evaluation of understanding or trust
NFR-07	Not fully verified	Browser tests for responsive layout and installability exist but were not rerun; no device matrix was tested
NFR-08	Met	Clean rebuild from an empty database; pinned dependencies; the model result reproduced on repeated runs
NFR-09	Met	205 backend tests pass with the documented environment
NFR-10	Not re-verified	Dashboards were inspected earlier, before the administrator pages were implemented, and were not inspected again for this chapter

5.7.4 Limitations

Table 5.21 lists the limitations that affect how far the findings can be trusted. None has been turned into a stronger claim.

Table 5.21 Limitations of the study

Limitation	**Effect on the findings**
Synthetic data	All 3,000 training records were simulated. The results describe the behaviour of the system on simulated Nairobi data and say nothing about real providers, and the feature importance reflects relationships that were built into the simulator.
Evaluation design	The split is random across bookings and not grouped by provider, so the test measures new bookings of known providers and not new providers. Only one split was held out, and the baselines are single-variable linear models.
Stored traffic patterns	Congestion comes from stored corridor and time-slot averages. The system has no live traffic feed and cannot react to an incident on a given day.
Target and outcome	The model predicts a synthetic reliability score. It does not predict arrival time, and no study measured missed appointments or client satisfaction.
No user study	Usability, accessibility and the clarity of the explanations were not tested with users.
Security	Authentication is not verified on the server, so the prototype must not be used with real accounts. Administrator verification reviews profile and contact details and does not check a government identity document. No deletion or retention policy exists (DR-08).
Environment	Performance was measured in one environment with sequential requests and no load test, and the hosted deployment was not independently verified.